

SI-AT-002 — Calling-Path Folding as a Recurring Computational Pattern

Series: Structural Intelligence — Additional Thoughts

Release Track: Post-Release Structural Delta

Status: Conceptual Extension and Research Hypothesis

Version Context: SI Compass v0.2.0

Sizhe Tan

with chatGPT (OpenAI)

2026-08-20

Repository: <https://github.com/sizhet/Structural-Intelligence-Compass>

Abstract

Major advances in computation do not always arise from discovering entirely new underlying capabilities.

Sometimes the breakthrough comes from **folding long, expensive, repetitive, or difficult computational paths into representations and mechanisms that make those paths dramatically easier to invoke.**

The positional numeral system provides an early and powerful example. Earlier numeral systems could represent quantities, but positional notation combined representation with a compact operational structure that made arithmetic substantially easier to perform and mechanize.

Modern computing repeatedly exhibits related patterns:

- algebra folds families of numerical relationships into symbolic forms;
- programming languages fold machine operations into reusable abstractions;
- functions and libraries fold implementation paths behind callable interfaces;
- APIs fold complex subsystems behind compact invocation boundaries;
- compiled artifacts fold construction and translation work into reusable runtime objects;

- Large Language Models fold enormous language, knowledge, and behavioral regularities into a parameterized system that can be invoked through relatively short runtime prompts.

These mechanisms are not mathematically identical.

Their commonality is structural:

Long or distributed computational paths become reachable through shorter reusable representations or invocation paths.

This article proposes **Calling-Path Folding** as a recurring computational pattern and explores its relationship to representation, operator economy, runtime cost, traceability, structural loss, and Structural Intelligence.

The hypothesis is that Calling-Path Folding is not merely a property of LLMs. It may represent a recurring mechanism in the history of computational civilization.

1. Representation Is Not the Same as Computation

A representation can describe an object without making operations over that object convenient.

This distinction is fundamental.

Consider a numeral system.

Its first requirement is straightforward:

Represent a quantity.

But another requirement is computational:

Make operations over quantities convenient, systematic, and reusable.

These are different requirements.

A representation may succeed at the first while performing poorly at the second.

Conceptually:

Representation

|
+----- Can the object be expressed?

|
+----- Can operations over the object be performed efficiently?

The second question introduces **computational affordance**.

A representation that exposes useful computational structure does more than record information.

It participates in computation.

2. Numerals as an Early Example

Roman numerals can represent quantities.

For example:

VIII

XIX

XLVII

The represented quantities are unambiguous.

However, consider:

XLVII × XXIII

The notation itself provides relatively little local structure for systematic multiplication.

The computation is possible.

The representation is simply not especially helpful.

Now consider positional decimal notation:

47 × 23

or:

3427

× 23

10281

68540

78821

The important difference is not merely that fewer characters are used.

The positional representation exposes:

- digit position;
- powers of the base;
- local arithmetic operations;
- carry;
- borrow;
- repeated operator patterns.

Representation and computation become tightly coupled.

3. Positional Representation as Computational Folding

A positional numeral system can therefore be viewed as more than a notation.

It provides a compact structure within which a large family of arithmetic operations can be executed through a small set of reusable procedures. Instead of requiring a distinct procedure for every large number, the system provides:

Position

+

Digit

+

Small Operator Set

|

v

Large Arithmetic Reach

The representation therefore contributes to an important form of **Operator Economy**.

A huge numerical problem space becomes accessible through repeated local operations.

This suggests a broader interpretation:

A powerful representation can fold large families of computational behavior into a compact operational structure.

This is an early form of the pattern explored in this article.

4. What Is Calling-Path Folding?

A computational calling path can be understood abstractly as a sequence of operations required to reach a result.

For example:

Input

|

v

Operation A

|

v

Operation B

|

v

Operation C

|

v

Operation D

|
v

Result

Suppose this path is repeatedly useful.

A system may construct an abstraction:

Input

|
v

F

|
v

Result

where F encapsulates, compiles, learns, caches, summarizes, or otherwise makes the longer path available through a shorter invocation.

Conceptually:

A -> B -> C -> D -> E

becomes

F

This does not necessarily mean that the internal computation disappears.

Rather:

The caller no longer needs to reconstruct the complete path explicitly.

That is the essential intuition behind **Calling-Path Folding**.

5. Folding Is Not Mere Compression

Calling-Path Folding should not be confused with ordinary data compression.

Compression primarily asks:

Can the same information be represented using fewer resources?

Calling-Path Folding asks:

Can a previously long or distributed computational process become accessible through a shorter reusable invocation structure?

The distinction can be represented as:

Data Compression

Large Representation

|
v

Small Representation

versus:

Calling-Path Folding

Long / Distributed Computational Path

|

v

Reusable Invocation Structure

|

v

Shorter Runtime Reachability

The two may overlap.

But their computational purposes differ.

Calling-Path Folding is fundamentally about **reachability and reuse**.

6. Algebra as Structural Folding

Algebra provides another example.

A large family of individual arithmetic relationships can be represented symbolically.

Instead of separately recording:

$$3 + 5 = 8$$

$$7 + 5 = 12$$

$$20 + 5 = 25$$

...

one may express:

$$x + 5$$

or more generally:

$$f(x)$$

The symbolic representation folds a family of concrete cases into a reusable structure.

This provides:

Many Instances

|

v

Symbolic Relation

|

v

Reusable Computation

Again, the important property is not simply shorter notation.

It is the creation of **computational generality**.

7. Functions as Calling-Path Folding

Programming makes the pattern explicit.

Consider:

A
|
B
|
C
|
D
|
E

If this sequence performs a coherent computational job, it can be encapsulated:

function F(...)

The caller then sees:

Input -> F -> Output

while the implementation may remain:

F
|
+----- A
|
+----- B
|
+----- C
|
+----- D
|
+----- E

The function creates a shorter calling path at the caller level.

This is one of the most fundamental organizational mechanisms in software engineering.

8. Libraries and APIs

Libraries extend the same principle.

A library may contain:

Algorithms

Data Structures

Validation

Memory Management

Error Handling

Optimization

Platform-Specific Logic

The application may see only:

library.call(...)

An API can fold an even larger subsystem:

Application

|

v

API Call

|

v

+-----+

| Authentication |

| Routing |

| Data Access |

| Computation |

| Distributed Services |

| Validation |

| Logging |

| Persistence |

+-----+

|

v

Response

The runtime caller does not reproduce these internal paths.

It invokes them.

Thus software engineering continuously converts:

implementation complexity into callable structure.

9. Compilation as Temporal Folding

Compilation reveals another dimension.

Consider:

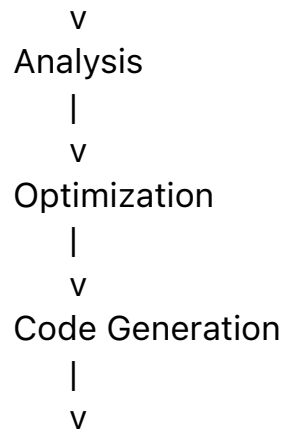
Source Program

|

v

Parsing

|

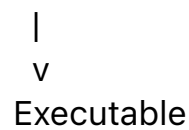


Executable

The expensive transformation occurs before repeated runtime execution.

After compilation:

Input



Executable

Output

This can be interpreted as a form of **temporal folding**.

Work is moved from repeated runtime execution into an earlier construction phase.

The runtime path becomes cheaper because previous computation has been embodied in a reusable artifact.

This distinction becomes especially important when considering trained AI models.

10. Training as Construction-Time Computation

Machine learning exhibits a related structure.

During training:

Large Dataset



Repeated Optimization

Parameter Updates



Trained Model

At runtime:

Input

|

v

Model

|

v

Output

The runtime invocation benefits from computational work performed earlier.

The model therefore acts as a reusable computational object constructed from prior observations and optimization.

This does not imply that the model stores literal copies of all training paths. It means that training creates a structure through which learned regularities become cheaply reusable at runtime.

11. LLMs as Large-Scale Calling-Path Folding

Large Language Models push this pattern to an extraordinary scale.

Before modern LLMs, many language-intelligence tasks required different combinations of:

Reading

Searching

Remembering

Comparing

Summarizing

Translating

Rephrasing

Pattern Recognition

Domain Knowledge

Example Retrieval

Language Generation

These activities were performed by humans, specialized software, databases, search systems, handcrafted rules, or combinations of them.

LLM training exposes a model to enormous amounts of language and optimizes a parameterized system capable of reproducing and generalizing many linguistic and knowledge-related regularities.

A simplified view is:

Massive Language Corpus

|

v

Training and Optimization

|
v

Large Parameterized Model

|
v

Prompt

|
v

Runtime Generation

From the perspective of the caller:

Prompt

|
v

LLM

|
v

Response

The runtime path is remarkably short relative to the scale of knowledge and linguistic structure that can influence the output.

This motivates the interpretation:

LLMs are large-scale Calling-Path Folding systems for language-mediated intelligence.

12. What Exactly Is Folded in an LLM?

Care is required here.

An LLM does not literally store a database of complete historical reasoning paths and replay one of them.

Nor does every parameter correspond to an identifiable linguistic rule or human concept.

The folding is distributed.

A better abstraction is:

Large Population of

Language Patterns

Knowledge Relations

Behavioral Regularities

Statistical Dependencies

Partial Computational Patterns

|
v

Distributed Parameterization

|
v

Runtime Conditional Generation

Thus the term **Calling-Path Folding** refers to the computational effect:

A large body of previously expensive or distributed language-related capability becomes reachable through a compact runtime interface.

It does not imply literal storage of original paths.

13. Folding and Runtime Economy

Calling-Path Folding produces a recurring economic pattern:

High Construction Cost

|
v

Reusable Structure

|
v

Lower Marginal Runtime Cost

Examples include:

Algorithm design

-> reusable algorithm

Compilation

-> executable

Library construction

-> library call

Index construction

-> fast lookup

Model training

-> inference

LLM training

-> prompt-based runtime access

This is a fundamental computational trade:

Pay more once to make future invocation cheaper.

The magnitude varies enormously across systems.

But the structural pattern recurs.

14. Operator Economy

Calling-Path Folding also connects naturally to **Operator Economy**.

A powerful computational system does not necessarily require a unique operator for every problem.

Instead:

Large Problem Space

|

v

Compact Representation

+

Small / Reusable Operator Set

|

v

Large Computational Reach

Positional arithmetic demonstrates this clearly.

Programming languages provide another example.

LLMs exhibit a different form:

Large Language-Intelligence Space

|

v

Parameterized Representation

+

Repeated Neural Operators

|

v

Large Behavioral Reach

The implementations differ.

The recurring principle is:

A compact computational substrate can expose a much larger effective capability space.

15. Folding and Structural Loss

Folding has benefits.

But it may also have costs.

When a long path is folded into a compact structure, some information about the original path may become difficult or impossible to recover.

This introduces an important distinction.

Explicit Folding

Examples include:

- functions;
- symbolic algebra;
- many algorithms;
- explicit calling graphs.

The folded structure often retains clear relationships to its internal organization.

Implicit or Distributed Folding

Examples include many learned models.

The relationship between the runtime result and the historical structures contributing to that result may be difficult to reconstruct.

Thus:

Folding

```

|
+---- Runtime Economy
|
+---- Reuse
|
+---- Generalization
|
+---- Possible Loss of Traceability

```

The stronger the folding, the more important the question becomes:

What structural information remains recoverable?

16. Folding and Unfolding

This leads to a useful paired concept:

FOLDING

Long Paths

```

|
v

```

Compact Runtime Structure

versus:

UNFOLDING

Compact Runtime Result

```

|
v

```

Recoverable Structural Path

Many conventional software abstractions support strong unfolding.

A function call can often be traced into its implementation.

A compiled program can often be debugged using mappings back to source structures.

A calling graph can expose possible paths.

LLMs are different.

An LLM may generate an explanation of its answer, but such an explanation should not automatically be interpreted as a faithful reconstruction of the internal causal computational path that produced the answer.

Therefore:

Generated explanation is not necessarily computational unfolding.

This distinction is central to the Structural Intelligence interpretation of LLMs.

17. Arabic Numerals and LLMs: Similarity and Boundary

The comparison between positional numerals and LLMs is useful precisely because the mechanisms are very different.

Positional numerals provide:

Explicit Representation

+

Explicit Operators

+

Strong Local Structure

+

High Traceability

LLMs provide:

Distributed Representation

+

Learned Parameters

+

Emergent Runtime Behavior

+

Weak Direct Path Reconstruction

They should therefore not be treated as mathematically equivalent systems.

Their structural similarity lies elsewhere:

Both represent major transitions in which large classes of previously cumbersome operations became accessible through dramatically more convenient computational structures.

For arithmetic:

Quantity

|
v
Positional Representation

|
v
Systematic Arithmetic
For language-mediated intelligence:
Large Language / Knowledge Space

|
v
Learned Parameterization

|
v
Prompt-Accessible Runtime Capability
The analogy concerns **computational accessibility**, not implementation identity.

18. A Recurring Pattern in Computational Civilization

Calling-Path Folding can be used as a lens through which to view several historical developments:

Writing

Experience / Knowledge

->

Symbolic Persistence

Positional Numerals

Quantitative Operations

->

Operational Number Representation

Algebra

Families of Relationships

->

Symbolic Generalization

Programming Languages

Machine Operations

->

Reusable Procedural Representation

Functions and Libraries

Implementation Paths

->

Callable Components

APIs

Complex Systems

->

Invocation Boundaries

Compilation

Construction-Time Work

->

Reusable Runtime Artifact

Machine Learning

Historical Data and Optimization

->

Reusable Learned Function

LLMs

Large Language-Intelligence Space

->

Prompt-Accessible Parameterized Runtime

These systems are heterogeneous.

Yet they repeatedly perform a related civilizational function:

They reduce the runtime burden required to reach useful computation.

19. Folding Changes What Becomes Economically Computable

This point deserves emphasis.

A computation may be theoretically possible but practically inaccessible because its calling path is too expensive.

Calling-Path Folding can change this.

Before folding:

Useful Result

^

|

Long / Expensive / Specialized Path

|

|

User

After folding:

User

|

v

Short Invocation

|

v

Reusable Structure

|

v

Useful Result

The result may not be new.

What changes is **reachability**.

This suggests:

Computational progress is sometimes less about making the impossible possible than making the expensive routinely reachable.

That is one reason Calling-Path Folding can have enormous economic and social effects.

20. LLMs and the Accessibility of Language Intelligence

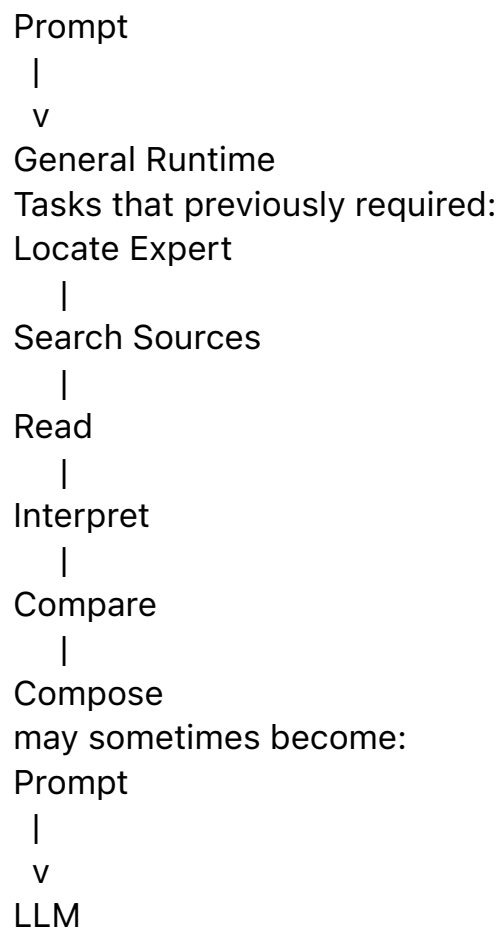
This perspective provides a different way to understand the importance of LLMs.

LLMs need not be interpreted as having invented language intelligence.

Human civilization already possessed:

- language;
- reasoning;
- writing;
- accumulated knowledge;
- translation;
- explanation;
- argument;
- abstraction;
- programming;
- scientific literature.

The breakthrough is partly that a remarkable portion of this space has become accessible through:



The reduction in invocation cost is itself transformative.

21. Why Folding Does Not Eliminate Structure

A successful folded system can create the impression that the underlying structure no longer matters.

This is dangerous.

A short invocation:

Prompt -> Answer

may hide:

Knowledge Structure

Language Structure

Training Structure

Representation Structure

Model Structure

Runtime Structure

Tool Structure

Validation Structure

Folding reduces what the caller must explicitly traverse.

It does not necessarily eliminate the underlying computational organization.

This produces a central Structural Intelligence question:

After computation has been folded, how can useful structure be recovered, localized, organized, validated, and selectively invoked?

This is where Structural Intelligence and LLMs become complementary rather than competitive.

22. Folding and Structural Intelligence

Structural Intelligence can be viewed as operating around folded computational systems.

A simplified relationship is:

Large Computational Space

|

v

Folding

|

v

Compact Computational Resource

|

v

Structural Organization

|

v

Selective Calling

|

v

Runtime Execution

|

v

Validation

The LLM may provide a powerful folded resource.

Structural Intelligence may help determine:

- where that resource belongs;
- when it should be invoked;
- which local structure should constrain it;
- which alternative computational instrument should be preferred;
- how results should be validated;

- how runtime provenance should be preserved.

Thus:

Folding provides capability concentration.

Structural organization provides capability placement.

The two solve different computational problems.

23. From Competition to Complementarity

This distinction changes the framing of many AI debates.

The question need not be:

LLM

versus

Structural AI

A more productive question is:

What computational job does each perform?

For example:

LLM

-> large-scale folded language intelligence

Differential Tree

-> structural organization

CCC / Two-Way CCC

-> structural recognition and discrimination

Calling Graph

-> computational reachability

FTRI

-> runtime switching

Policy

-> computational selection

Validation

-> outcome verification

These are not necessarily competing claims to one computational throne.

They may be complementary components of a larger intelligence architecture.

24. Folding as a Source of Policy Surfaces

Once folded intelligence coexists with explicit structural intelligence, a new question appears.

Suppose a runtime node can choose among:

Algorithm

Local ANN

Two-Way CCC

LLM

Search

Human

Then the system must determine:

When should folded intelligence be used?

When should explicit structure be preferred?

When should both cooperate?

When should one validate the other?

This creates a **Policy Surface**.

For example:

Problem

|

v

Structural Localization

|

v

Policy

|

+----- Explicit Algorithm

|

+----- Local Learned Model

|

+----- Two-Way CCC

|

```

+----- LLM
|
+----- Combination
|
v

```

Runtime

Calling-Path Folding therefore does not terminate structural computation. It creates new opportunities for policy-controlled structural composition.

25. Dynamic Folding

Calling-Path Folding need not always occur through large preconstructed systems.

A runtime organization may discover that certain paths are repeatedly executed:

$A \rightarrow B \rightarrow C \rightarrow D$

If the path is stable and valuable, the system may eventually construct:

F

where:

$F \sim=$ reusable form of $A \rightarrow B \rightarrow C \rightarrow D$

The implementation might involve:

- caching;
- compilation;
- model training;
- local ANN construction;
- structural invariant extraction;
- Two-Way CCC formation;
- function generation;
- workflow packaging.

This suggests a future research direction:

Can Structural Intelligence identify when runtime paths should themselves be folded into new computational units?

This turns folding from a historical observation into an active runtime

mechanism.

26. Folding, Organization, and Evolution

A more complete cycle becomes possible:

Enumerate

|

v

Organize

|

v

Execute

|

v

Observe Repeated Paths

|

v

Fold

|

v

Create New Computational Unit

|

v

Reorganize

|

v

Execute Again

This suggests that computational organizations may evolve.

Repeated successful behavior can become structure.

Repeated structure can become a reusable primitive.

The primitive can then participate in higher-level organization.

Conceptually:

Runtime Experience

|

v

Stable Pattern

|

v

Folding

|

v

Computational Primitive

|
v

Higher-Level Runtime

This provides a possible bridge between **Structural Synthesis** and **Structural Evolution**.

27. A Research Classification of Folding

Future work may distinguish several forms of Calling-Path Folding.

27.1 Representational Folding

Large families of objects become accessible through compact representation.

Examples:

- positional numerals;
- algebraic notation.

27.2 Procedural Folding

Repeated operation sequences become callable units.

Examples:

- functions;
- libraries;
- workflows.

27.3 Interface Folding

Complex subsystems become reachable through stable invocation boundaries.

Examples:

- APIs;
- services.

27.4 Temporal Folding

Expensive work is moved into an earlier construction stage.

Examples:

- compilation;
- indexing;

- model training.

27.5 Learned Folding

Statistical or structural regularities become embedded in learned computational objects.

Examples:

- ANN;
- learned policies;
- LLMs.

27.6 Structural Folding

Repeated explicit computational structure becomes a reusable structural primitive.

Possible examples include:

- reusable calling paths;
- invariant structures;
- local CCC;
- runtime-generated Brain Units.

These categories may overlap.

Their purpose is not rigid taxonomy but research guidance.

28. A Testable Research Direction

Calling-Path Folding should eventually become measurable.

Possible measures include:

Original Path Length

Folded Invocation Length

Construction Cost

Runtime Cost

Reuse Frequency

Accuracy

Structural Loss

Traceability

Unfolding Cost

Adaptability

Validation Cost

A generic measure could compare:

Before Folding

Runtime Cost = $C1$

Path Complexity = $P1$

Traceability = $T1$

After Folding

Construction Cost = Cf

Runtime Cost = $C2$

Invocation Complexity = $P2$

Traceability = $T2$

with:

$C2 < C1$

$P2 < P1$

while examining what happens to:

$T2$

and other structural properties.

This could turn Calling-Path Folding from a descriptive metaphor into an experimentally useful computational concept.

29. Boundary of the Claim

This article does **not** claim that:

- positional numeral systems and LLMs are mathematically equivalent;
- every abstraction is Calling-Path Folding;

- compression and Calling-Path Folding are identical;
- LLM parameters contain explicit copies of human reasoning paths;
- folded computation necessarily preserves original computational provenance;
- folding is always beneficial.

The narrower claim is:

Across many computational systems, long, distributed, repeated, or expensive computational paths are transformed into reusable structures that permit shorter or cheaper runtime invocation.

This recurring pattern is called **Calling-Path Folding**.

The hypothesis is that studying this pattern explicitly may help explain both historical computational advances and future Structural Intelligence architectures.

30. From Folding to Structural Synthesis

Calling-Path Folding changes the available computational landscape.

Once a powerful folded structure exists, the next problem becomes organizational:

What should call it?

When?

From where?

Under which policy?

With which context?

At what cost?

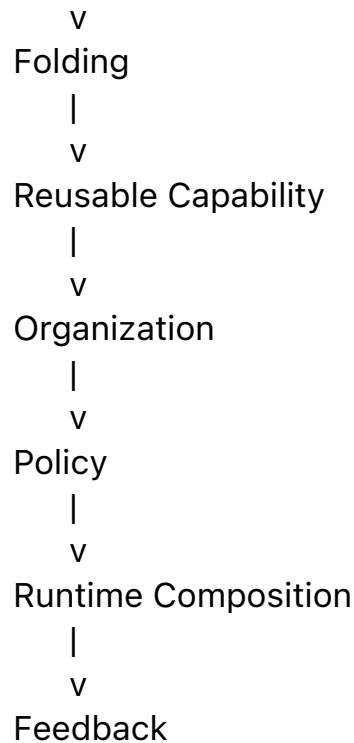
With what fallback?

With what validation?

Thus the computational progression may be:

Representation

|



This provides a direct bridge from LLMs to the broader Structural Intelligence research program.

31. Conclusion

The history of computation repeatedly demonstrates that major progress can come from making existing capability easier to invoke.

Positional numerals made arithmetic structurally convenient.

Algebra made families of relationships symbolically reusable.

Functions and libraries made procedural paths callable.

APIs made complex systems accessible through compact interfaces.

Compilation moved repeated work into reusable artifacts.

Machine learning converted historical data and optimization into reusable learned computation.

Large Language Models have carried this pattern into language-mediated intelligence at unprecedented scale.

The implementations are different.

The recurring computational pattern is:

Large / Long / Distributed Computational Space



Reusable Computational Structure

|

v

Short Invocation

|

v

Runtime Capability

Calling-Path Folding therefore provides one possible structural explanation for why certain representations and computational technologies have produced disproportionate changes in human capability.

The next question is equally important:

Once powerful computation has been folded, how should it be structurally organized, selectively invoked, validated, and combined with other forms of intelligence?

That question moves beyond folding itself.

It leads toward **Structural Synthesis**.

Closing Perspective

Arabic numerals did not create quantity.

They made quantitative computation radically more accessible.

Programming languages did not create machine operations.

They made those operations structurally reusable.

APIs did not create the systems behind them.

They made those systems cheaply callable.

LLMs did not create human language, accumulated knowledge, or the underlying phenomena expressed through language.

They have made an extraordinary range of language-mediated capability accessible through a compact runtime interface.

The recurring lesson is:

A civilization advances not only by discovering new computation, but also by folding existing computational paths into forms that become easier to reach.

Structural Intelligence adds the next question:

After the paths are folded, how should the resulting capabilities be organized?

That transition —

Folding

->

Organization

->

Policy

->

Runtime

->

Evolution

— may define an important part of the next computational frontier.